

Laboratorio di Ingegneria dell'Informazione - Modulo 3

A.A. 2025/2026

Fast Fourier Transform (FFT)

Background teorico. La FFT è un algoritmo che calcola la DFT con complessità ridotta, pari a $\mathcal{O}(N \log_2 N)$, scomponendo il problema in sottoproblemi di dimensione minore. Nel caso $N = 2^\ell$ si parla di FFT radix-2. Consideriamo, in particolare, la FFT radix-2 a decimazione in frequenza (DIF). Ricordiamo che la DFT è definita come

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{kn}$$

dove $W_N = e^{-j2\pi/N}$ è detta costante di rotazione (*twiddle constant*). L'algoritmo FFT radix-2 DIF si basa sull'osservazione che, per ogni $p = 0, \dots, N/2 - 1$, risulta:

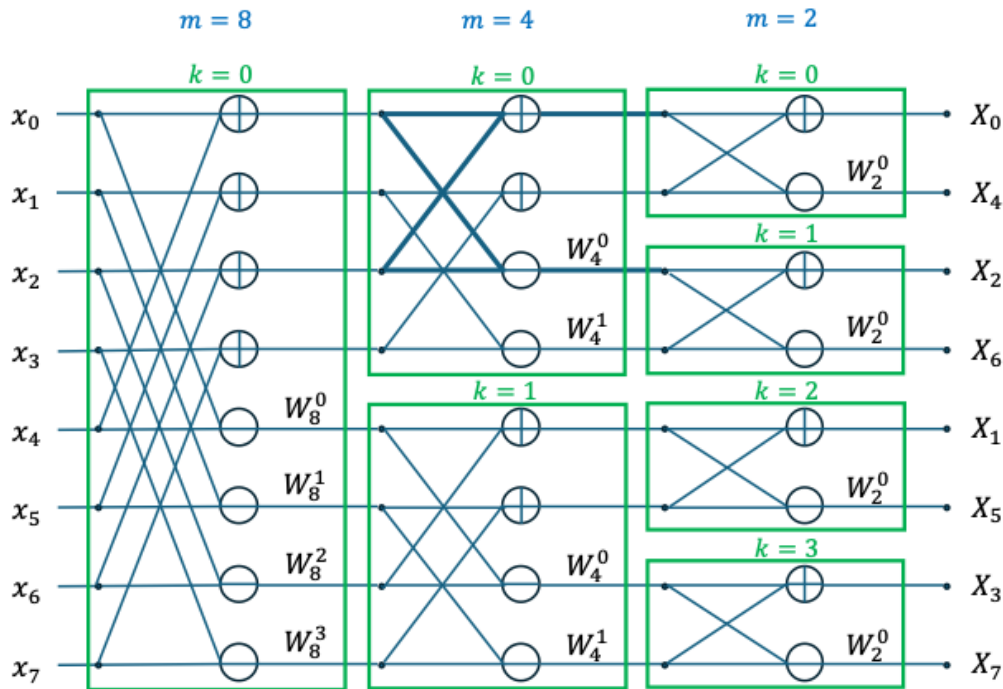
$$X[2p] = \sum_{n=0}^{N/2-1} (x[n] + x[n + N/2]) \cdot W_{N/2}^{np}$$
$$X[2p + 1] = \sum_{n=0}^{N/2-1} (x[n] - x[n + N/2]) \cdot W_N^n \cdot W_{N/2}^{np}$$

Possiamo dare al risultato la seguente interpretazione. Le espressioni di $X[2p]$ e $X[2p + 1]$ hanno entrambe la struttura matematica di una DFT a $N/2$ punti. Questo significa che, partendo dal vettore x , possiamo costruire due vettori di lunghezza $N/2$ e ricondurre il calcolo del vettore X al calcolo delle DFT di questi due vettori, cioè al calcolo di due DFT a $N/2$ punti anziché di una DFT a N punti. La FFT radix-2 DIF si ottiene iterando il procedimento fino ad arrivare a DFT a 2 punti. Sempre nel caso $N = 8$, si ottiene lo schema mostrato nella figura alla pagina seguente.

Per implementare la FFT radix-2 DIF possiamo osservare quanto segue:

- L'elaborazione è suddivisa in $\log_2 N$ stadi, che indicizziamo con $m = N, N/2, N/4, \dots, 2$. Ad esempio, nel caso dello schema in figura abbiamo $N = 8$, quindi $\log_2 8 = 3$ stadi con indici $m = 8, 4, 2$.
- Lo stadio m comprende N/m gruppi, che indicizziamo con $k = 0, 1, \dots, N/m - 1$. I blocchi sono evidenziati in verde nello schema in figura.
- In ciascun gruppo dello stadio m vengono calcolate $m/2$ "butterfly", ciascuna delle quali comprende una somma, una sottrazione e una moltiplicazione (vedere struttura evidenziata in figura). Possiamo indicizzare le butterfly del gruppo con $j = 0, \dots, m/2 - 1$.
- Ogni stadio ha N ingressi, indicizzati da 0 a $N - 1$. Considerato lo stadio m e il gruppo k , la butterfly di indice j elabora gli ingressi con indici $(m k + j)$ e $(m k + j + m/2)$.

Bit reversal. Come si può osservare dalla figura, gli elementi del vettore trasformato non sono in ordine. Essi possono essere ordinati eseguendo una procedura nota come *bit reversal*, che consiste nel considerare la rappresentazione binaria degli indici degli elementi del vettore di uscita e nell'invertire l'ordine di tali bit. Esempio: con riferimento alla figura, l'uscita di indice $3_{10} = 011_2$ dovrà essere spostata nella posizione $110_2 = 6_{10}$.



Parte 1. Scrivere una funzione C

```
void butterfly(const double complex *a, double complex *b, double complex W)
```

che implementi la butterfly elementare, nella quale il primo parametro (a) è il vettore di ingresso di lunghezza 2, il secondo parametro (b) è il vettore di uscita di lunghezza 2 e il terzo parametro è la costante di twiddle della butterfly.

Parte 2. Utilizzando le convenzioni sugli indici sopra introdotte, scrivere una funzione C

```
void fft(const double complex *x, double complex *X, int N)
```

che implementi la FFT radix-2 DIF senza il riordino degli elementi del vettore trasformato. Tale funzione avrà un ciclo esterno (in m) sugli stadi, un ciclo intermedio (in k) sui gruppi e un ciclo più interno (in j) sulle butterfly. Nella funzione si consiglia di definire due array, $in[N]$ e $out[N]$, che rappresentano rispettivamente l'ingresso e l'uscita del generico stadio¹; al termine dello stadio, il vettore out viene copiato nel vettore in .

Parte 3. Completare la funzione `fft` aggiungendo l'ultimo stadio di bit reversal. E' possibile utilizzare direttamente la funzione proposta alla pagina seguente.

Homework. Riprendere l'esercitazione 5, "Trasformata Discreta di Fourier", ed eseguire le parti 1 e 2 di tale esercitazione sostituendo la funzione `dft` con la funzione `fft` oggetto della presente esercitazione, osservando come gli spettri non cambino.

¹ Si tratta di due VLA ("variable length array"), cioè array la cui dimensione non è una costante.

```

void bit_reversal(const double complex *out, double complex *X, int
N)
{
    /* nbits = log2(N) = numero bit per label */
    int nbits = 0, N1 = N;
    while (N1 >>= 1) nbits++;

    for (int i = 0; i < N; i++) {
        int v = i;
        int r = 0;

        /* Rappresentazione di r = quella di v con i bit invertiti */
        for (int j = 0; j < nbits; j++) {
            int bit = v % 2;
            r = r * 2 + bit;
            v = v / 2;
        }

        X[r] = out[i];
    }
}

```